241080076 Vishwajeet Waghmare | 241080077 Awwab Wadekar
241080078 Yash Wadhwa | 241080082 Abiraj Kangotra

Hybrid Offline-First HMS: Project Estimation

7.Software Design Modeling & Specification Document

1. Executive Overview & Design Scope

Design Context: This document translates the functional requirements of the **Hybrid Offline-First Hospital Management System (HMS)** into explicit technical design models. It covers database physical schemas, system architecture, transform/transaction mapping charts, user interface layout wireframes, and component procedural algorithms.

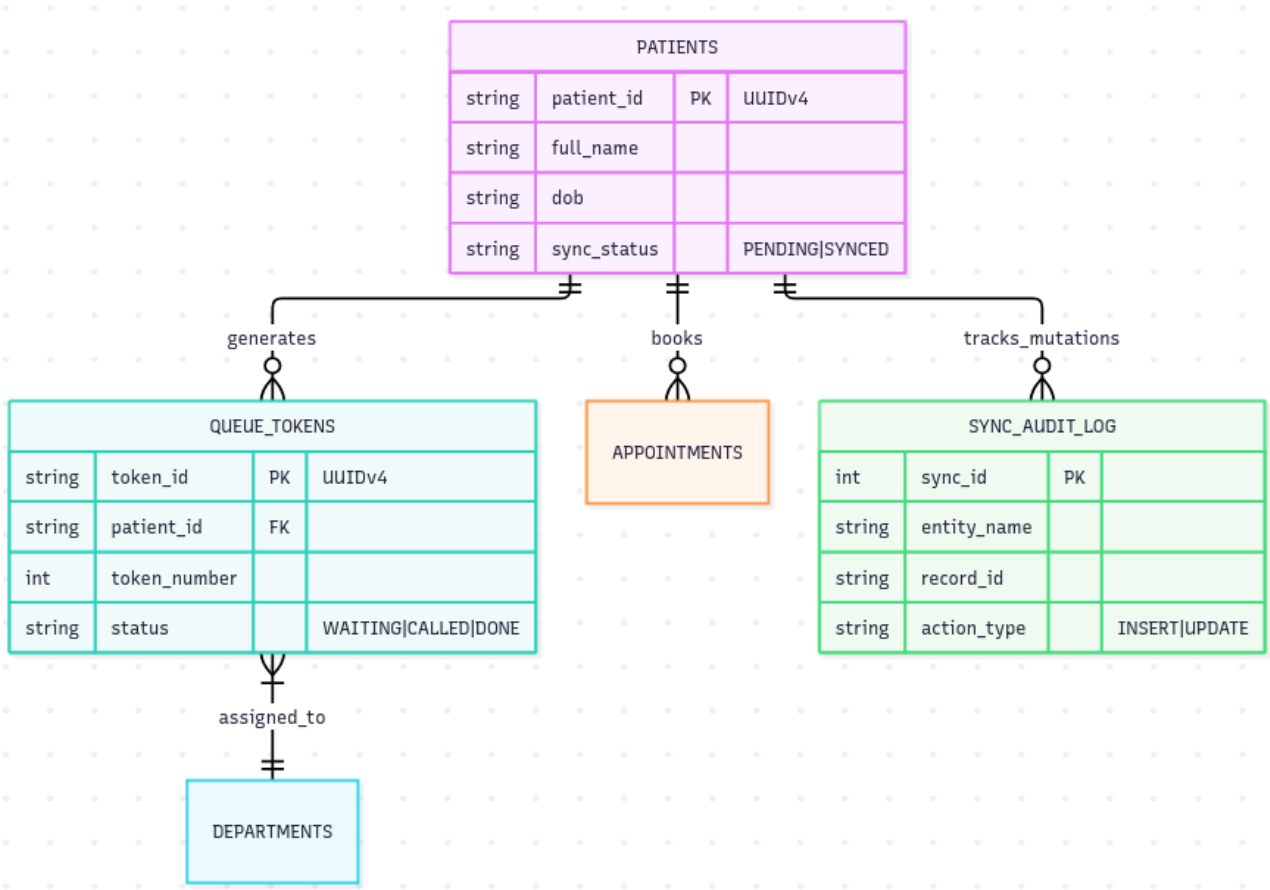
2. Data Design (Storage Schemas & Data Structure Transformations)

Data design maps conceptual entity models directly into physical SQLite database tables optimized for local performance and cloud sync audit tracking:

Table Name	Columns, Data Types & Keys	Integrity Constraints	Performance & Storage Optimization
patients	patient_id (UUID, PK), name (TEXT), dob (DATE), gender (VARCHAR), created_at (TIMESTAMP)	NOT NULL on name/dob; Unique UUID key.	Indexed on patient_id and name for sub-10ms local lookups.
queue_tokens	token_id (UUID, PK), patient_id (FK), dept_id (INT), token_num	FK references patients(patient_id).	Composite B-Tree index on (dept_id, status) for fast display queries.

	(INT), status (TEXT)		
sync_audit_log	sync_id (INT, PK AUTO), entity (TEXT), record_id (UUID), action (TEXT), sync_status (TEXT)	CHECK (action IN ('INSERT','UPDATE','DELETE')).	SQLite WAL (Write-Ahead Logging) mode enabled to prevent read/write locks.

Data Design Diagram Code (Mermaid Entity-Relationship Diagram)

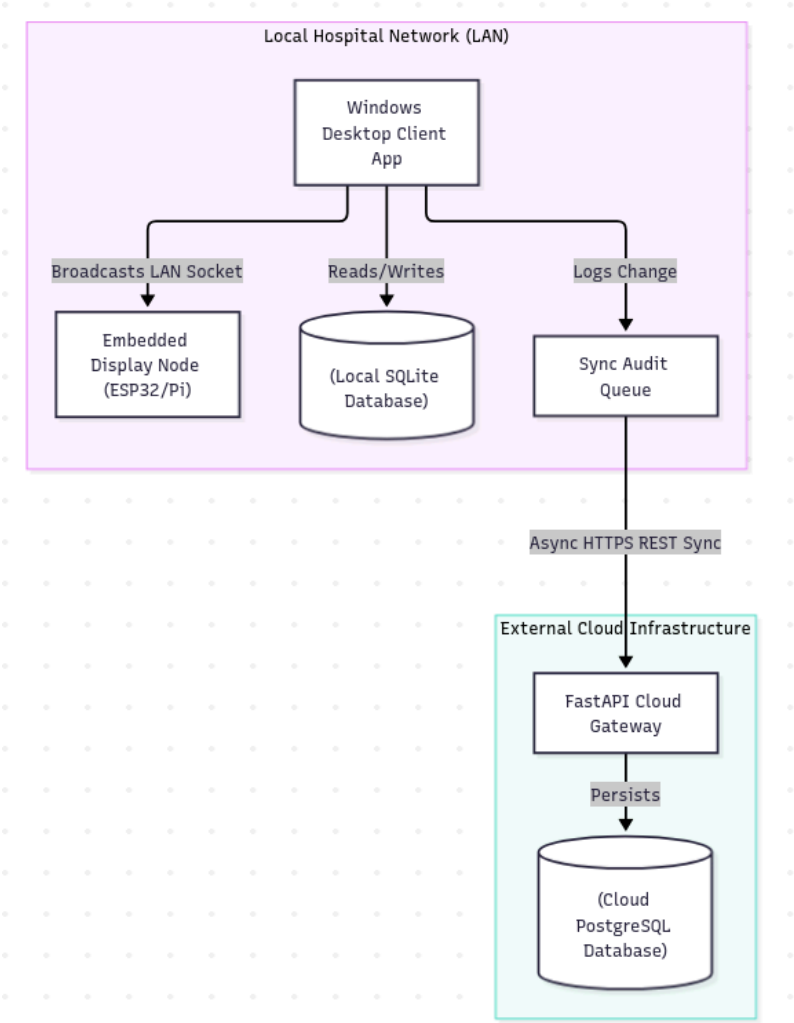


3. Architectural Design (Subsystem Topology & Structural Patterns)

The application employs a multi-tiered hybrid offline-first architecture:

1. **Presentation Layer:** WPF/PyQt desktop client (.exe) for reception/doctors and an embedded Pi/ESP32 display node software client.
2. **Local Persistence & Processing Layer:** Python business controllers, local SQLCipher encrypted SQLite database, and UDP/Socket LAN broadcast modules.
3. **Cloud Synchronization Layer:** Asynchronous HTTP/REST background worker, FastAPI gateway, and cloud PostgreSQL database.

Architectural Topology Diagram Code (Mermaid Flowchart)



4. Transform Mapping vs. Transaction Mapping

Translating DFD data flows into structural module hierarchies using formal design mapping:

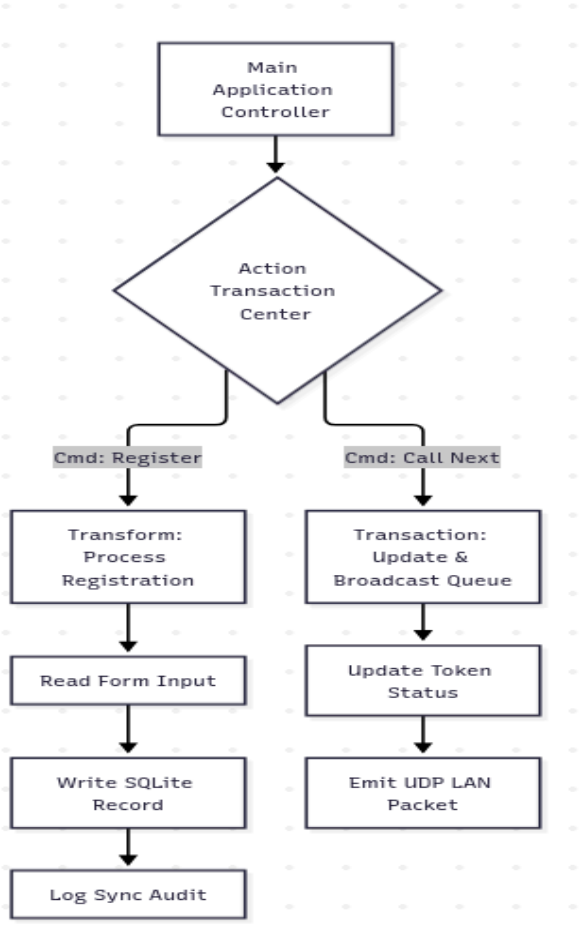
A. Transform Mapping (Sequential Data Flows)

- **Application Scope:** Used for linear processing pipelines such as Patient Registration and Offline Sync Flushing.
- **Incoming Center:** Captures raw UI form inputs and constructs validated memory models.
- **Transform Center:** Handles UUID generation, SQLite transactional writes, and sync audit logging.
- **Outgoing Center:** Produces printed token slips or constructs HTTP JSON sync request payloads.

B. Transaction Mapping (Branching & Event-Driven Flows)

- **Application Scope:** Used for event dispatchers such as Doctor Dashboard actions.
- **Transaction Center:** Evaluates incoming user button events and routes execution to appropriate action branches.
- **Action Branch 1 (Call Token):** Updates token state in SQLite and triggers immediate local socket broadcast to display nodes.
- **Action Branch 2 (Complete Consultation):** Generates digital prescription records and archives token.

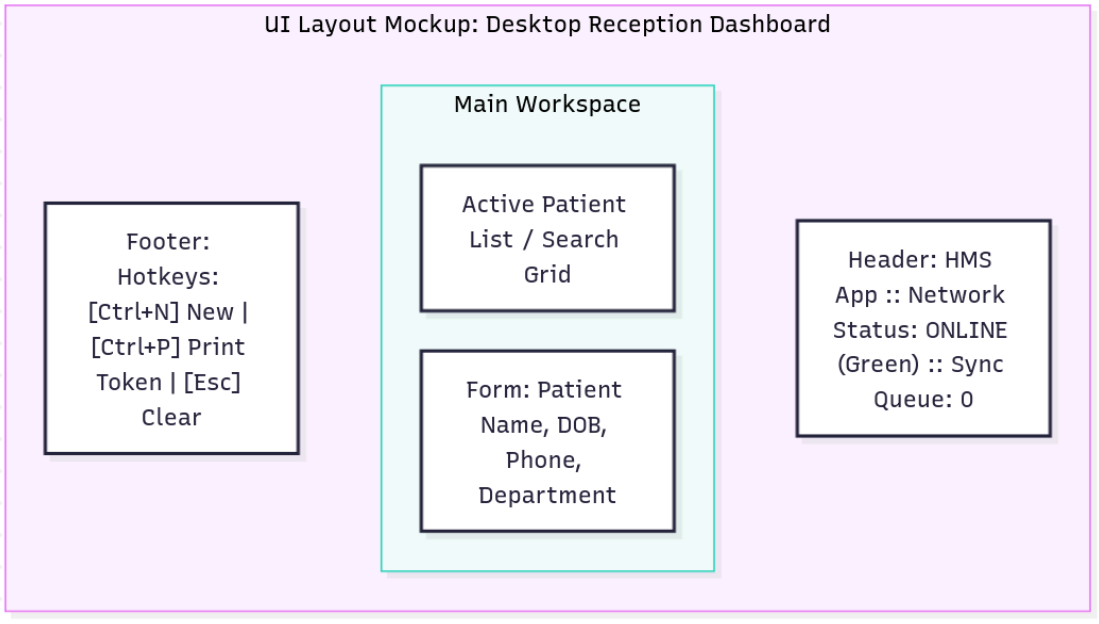
Transaction & Transform Mapping Diagram Code (Mermaid Structure Chart)



5. User Interface (UI) Design & Visual Mockups

UI Screen View	Primary Components & Controls	Ergonomics & Hotkey Actions	Visual Feedback & System Indicators
Reception Check-In View	Form inputs, Department dropdown, Auto-complete search, "Issue Token" CTA.	Ctrl+N (New), Enter (Submit); full keyboard navigation.	Instant green toast notification; automatic local print trigger.
Doctor Calling Console	Patient queue table, "Call Next" action button, Vitals/Notes text box.	Single-click calling; keyboard shortcut support.	Selected row highlighted; embedded display screen chimes.
Sync Status Bar	Online/Offline status badge, Pending audit counter, Retry button.	Tooltip hover for sync log details.	Color badge: Blue (Synced), Orange (Offline Cache), Red (Conflict).

UI Layout Wireframe Diagram Code (Mermaid Diagram)



6. Procedural / Component-Level Design

Procedural design details internal module logic using Structured Algorithms and Logic Flowcharts:

A. Component Algorithm: Offline Queue Token Generator

```
ALGORITHM IssueQueueToken(patient_data, department_id):
    INPUT: patient_data (Struct), department_id (Integer)
    OUTPUT: token_object (Struct), broadcast_status
(Boolean)

    BEGIN

        IF patient_data.patient_id IS NULL THEN
            patient_data.patient_id = GenerateUUIDv4()

            EXECUTE SQL
                "INSERT INTO patients VALUES (...)"
        ENDIF

        current_date = GetCurrentDateISO()

        max_num = EXECUTE SQL
            "SELECT MAX(token_num)
             FROM queue_tokens
             WHERE dept_id = department_id
             AND date = current_date"

        next_token_num = (max_num IS NULL)
                         ? 1
                         : max_num + 1

        token_id = GenerateUUIDv4()
```

```
    EXECUTE SQL
        "INSERT INTO queue_tokens VALUES
         (token_id, patient_data.patient_id,
          department_id, next_token_num, 'WAITING')"

    EXECUTE SQL
        "INSERT INTO sync_audit_log VALUES
         ('queue_tokens', token_id,
          'INSERT', 'PENDING')"

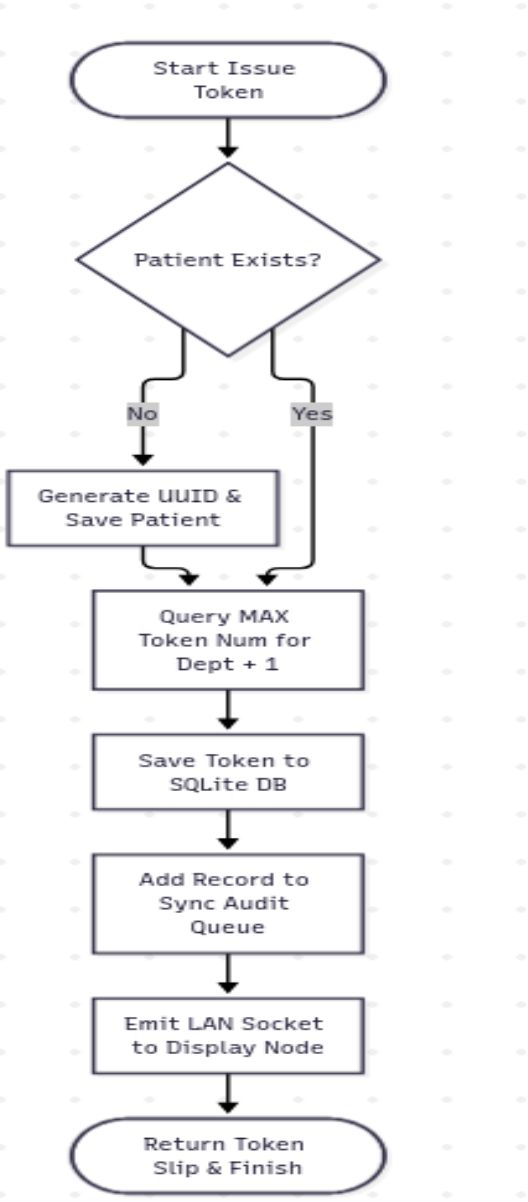
    payload = ConstructSocketJSON(
        token_id,
        next_token_num,
        department_id
    )

    broadcast_status = BroadcastToLAN(
        payload,
        port = 5005
    )

    RETURN Struct(
        token_id,
        next_token_num
    ), broadcast_status

END
```

B. Component Flowchart Diagram Code (Mermaid Flowchart)



7. Conclusion

This Software Design Modeling specification completes the transition from requirements analysis to concrete architectural, database, and procedural blueprints for the Hybrid Offline-First Hospital Management System. By detailing Data Design (UUID-driven local schemas), multi-tier Architectural Design, Transform/Transaction Mapping models, ergonomic UI layouts, and explicit component algorithms, the design guarantees long-term maintainability, sub-10ms query execution, and reliable offline operation. The included Mermaid markdown code blocks enable rapid rendering of publication-grade visual diagrams using accessible online tools.